Systems Track

Interview Preparation Series • Professional Edition

Operating Systems

02

Compiled with LaTeX • June 15, 2026

harsha.somisetty01@gmail.com

In This Edition

> Build from first principles. Understand deeply. Recall under pressure.

1 Overview --- What It Is

An **Operating System** is the layer of software that sits between hardware and user applications. It virtualizes physical resources (CPU, RAM, disk, network) into clean abstractions that programs can use without knowing hardware details.

Think of it as a **resource manager + referee + abstraction layer**:

- **Resource Manager**: decides who gets the CPU next, how RAM is divided, which process can write to disk
- **Referee**: enforces isolation so one process can't corrupt another
- **Abstraction Layer**: turns raw hardware into clean primitives — files, threads, sockets, processes

Core OS abstractions:

Physical Resource	OS Abstraction
CPU	Process / Thread
RAM	Virtual Address Space
Disk	File System
Network card	Socket
I/O devices	Device drivers / file-like interfaces

2 Why It Exists

Without an OS, every program would need to:

1. Speak directly to hardware (CPU-specific assembly, device-specific registers)
2. Manually share the CPU — programs would run one at a time forever
3. Trust every other program not to stomp on its memory
4. Re-implement disk access from scratch

Core problems the OS solves:

1. **Multiplexing** — many programs share one CPU (time-sharing) and one RAM (space-sharing)
2. **Isolation** — a buggy program crashes itself, not the whole system
3. **Abstraction** — programs deal with files, not spinning disk sectors
4. **Protection** — user code can't access kernel memory or hardware directly

The OS achieves this through **privileged hardware modes** and **virtualization**.

3 Why FAANG Cares (Specifics)

Google:

- Google's entire infrastructure runs on Linux. Engineers tune kernel scheduling (CFS), memory allocators (tcmalloc), and container isolation (cgroups/namespaces). Borg/Kubernetes

orchestration is fundamentally process/thread scheduling at cluster scale. Interview questions on `fork()`, copy-on-write, and memory mapping come up in systems design for search infrastructure.

Meta:

- Meta runs millions of PHP/Hack processes. HHVM relies on JIT and careful memory management. Their data centers run custom Linux kernels. Systems engineers are quizzed on IPC (shared memory for Scribe), page faults, and TLB shootdowns at scale.

Amazon (AWS):

- EC2 virtualizes physical machines — fundamentally OS concepts (hypervisors, page tables, context switching). Lambda's micro-VM (Firecracker) is a stripped-down kernel. S3/EBS performance hinges on OS-level I/O scheduling and file system design. Interview focus: virtual memory, file systems, system calls.

Apple:

- macOS/iOS are Darwin-based (XNU kernel). Apple engineers work on mach_vm, Grand Central Dispatch (GCD) which sits atop pthreads and kernel queues. Interview focus: Mach IPC, Mach-O file format, memory management (ARC interacting with VM system), kernel extensions.

Microsoft:

- Windows NT kernel engineers and Azure hypervisor team live in OS concepts daily. Windows Subsystem for Linux (WSL2) runs a full Linux kernel in a VM. Interview topics: NT kernel scheduling, memory mapped files, COM object model's reliance on processes/threads.

Uber:

- Uber's dispatch system processes millions of location updates/second. They've written extensively about tuning Linux kernel parameters (TCP stack, socket buffers, scheduler affinity). Engineers tune `SO_REUSEPORT`, `epoll`, and real-time scheduling for driver matching.

Databricks:

- Apache Spark runs JVM-based distributed computation. Understanding JVM GC (which is application-level memory management layered on OS virtual memory), huge pages (`madvise`), and NUMA topology affects Spark performance at scale. Engineers optimize shuffle I/O, which touches OS page cache, `sendfile()`, and direct I/O.

The interview reality: FAANG systems design rounds assume you understand what the OS does under the hood. When you say "cache this in memory," you should know what that means in terms of virtual memory, page cache, and eviction policies.

4 Core Concepts

4.1 Kernel vs User Mode

The CPU has at least two privilege levels (x86 has 4 rings, Linux uses ring 0 and ring 3):

```
Ring 3 (User Mode)  — your program lives here
    |
    |  system call (trap/interrupt)
    v
Ring 0 (Kernel Mode) — OS kernel lives here
    |
    v
Hardware (CPU, RAM, I/O)
```

User Mode:

- Restricted instruction set — can't directly access hardware, can't disable interrupts
- Limited address space — only sees its own virtual address space
- Any illegal operation → CPU raises exception → kernel takes over → usually kills the process (segfault)

Kernel Mode:

- Full access to hardware, all memory, all CPU instructions
- Can modify page tables, configure interrupts, access I/O ports

System Call = the boundary crossing. A program calls `read()` → C library wrapper executes a `syscall` instruction → CPU switches to ring 0 → kernel runs the actual read logic → returns result → CPU switches back to ring 3.

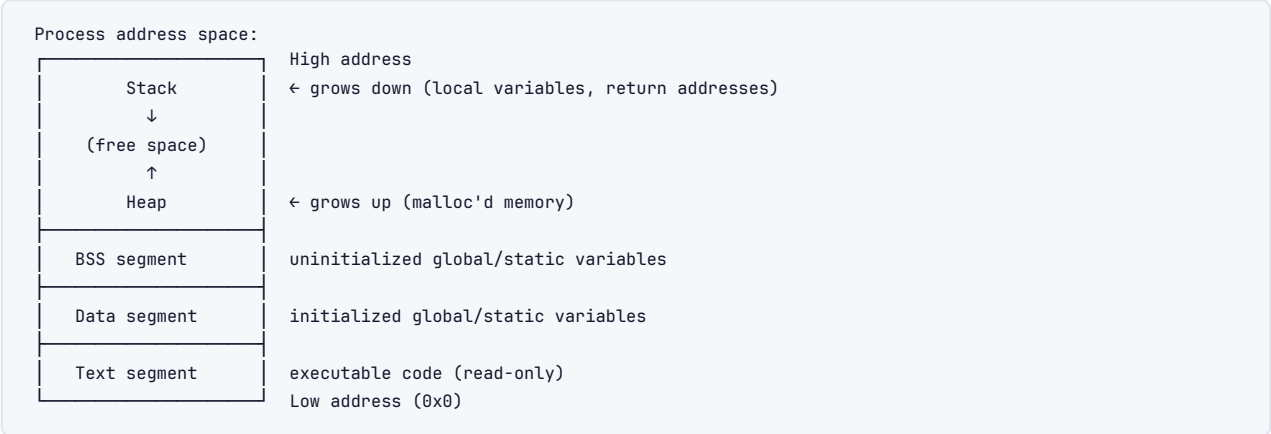
Cost of a system call: ~100-1000 ns. Crossing the boundary flushes parts of CPU pipeline, saves/restores registers. This is why programs that make many small syscalls are slower than ones that batch work (e.g., `write()` once with a big buffer vs. 1000 times with tiny buffers).

Interview takeaway: User mode protects the kernel from buggy/malicious user code. System calls are the only legal way to access kernel services.

4.2 Process vs Thread

Process

A process is an **instance of a running program** with its own isolated execution environment:



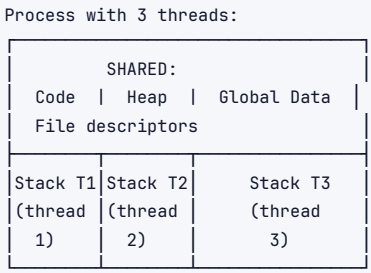
Each process has:

- **PID** (process ID)
- **Virtual address space** (its own illusion of contiguous memory)
- **Open file descriptors**
- **CPU registers** (saved when not running)
- **Privileges/credentials** (UID, GID)
- **Signal handlers**
- **Page tables** (mapping virtual → physical)

Thread

A thread is a **unit of execution within a process**. Multiple threads share the process address space but each has its own:

- **Stack** (each thread needs its own call stack)
- **Registers** (including program counter — where am I executing?)
- **Thread-local storage**



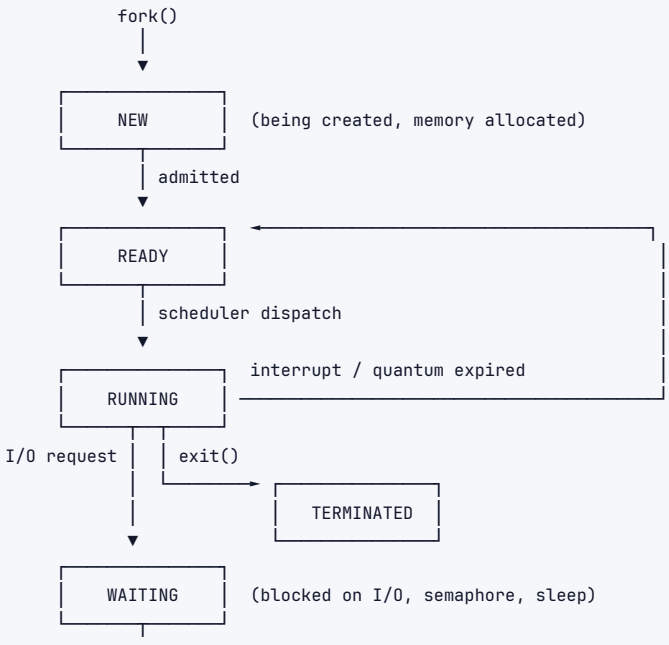
Key Differences Table

Aspect	Process	Thread
Memory space	Own virtual address space	Shared with other threads in process
Creation cost	High (copy page tables, file descriptors)	Low (just allocate stack + registers)
Communication	IPC (pipes, sockets, shared memory)	Direct (shared variables — but needs locks)
Isolation	Crash in one doesn't affect another	Crash (segfault) kills entire process
Context switch cost	High (flush TLB, switch page tables)	Low (same address space, partial register save)
Typical overhead	~MB	~few KB
Linux implementation	`fork()`	`clone()` with shared flags

Interview takeaway: Processes give isolation; threads give efficiency. Pick processes when you need fault isolation (Chrome uses a process per tab), threads when you need to share data cheaply and speed matters.

4.3 Process States and Lifecycle

A process moves through states as it runs, waits, and exits:



5 classic states: New → Ready → Running → Waiting → Terminated
Linux adds more nuance:

- `R` = Running or Runnable
- `S` = Interruptible Sleep (waiting, can be woken by signal)
- `D` = Uninterruptible Sleep (in kernel, usually waiting for I/O — can't be killed)
- `Z` = Zombie (finished but parent hasn't `wait()`d yet — PCB still exists)
- `T` = Stopped (paused by `SIGSTOP`)

Zombie Process: A process that has exited but whose exit status hasn't been collected by the parent yet. The PCB (Process Control Block) remains in the kernel. Fix: parent must call `wait()`.

4.4 fork() and exec()

fork(): Creates an exact copy of the calling process.

```c
pid_t pid = fork();
if (pid == 0) {
    // child process — pid == 0
    exec("/bin/ls", ...);
} else if (pid > 0) {
    // parent process — pid = child's PID
    wait(NULL);  // wait for child
} else {
    // error
}
```

What fork() copies:

- Virtual address space (using **copy-on-write**)
- File descriptors (same open files, shared file offset)
- Signal handlers
- Environment variables

What fork() does NOT copy:

- Memory locks
- Pending signals (cleared)
- Threads (only the calling thread exists in child — dangerous with multithreading!)

Copy-on-Write (CoW): After `fork()`, both parent and child share the same physical pages, marked read-only. When either tries to write, a page fault occurs, the kernel copies that page, and the writing process gets its own copy. This makes `fork()` fast when the child immediately calls `exec()`.

exec(): Replaces the current process's address space with a new program. The PID stays the same, but code, stack, and heap are completely replaced.

The classic pattern: `fork()` then `exec()` = create new process running a different program. This is how shells work.

vfork(): Like fork() but child borrows parent's address space (no copy, not even CoW). Parent is suspended until child calls exec() or exit(). Dangerous but useful for performance when you know exec() is next.

Interview takeaway: fork() + exec() is how Unix creates new processes. CoW makes fork() cheap. Zombie processes are cleaned up by parent calling wait().

4.5 Context Switching

When the OS switches from process A to process B:

Cost sources:

1. Direct: saving/restoring registers (~dozens of ns)
2. Cache pollution: process B's working set evicts A's data from CPU cache
3. TLB flush: new address space = all TLB entries invalid = many page table walks needed initially

Thread context switches are cheaper: same address space → no page table switch → no TLB flush.

PCB (Process Control Block) = kernel data structure that stores everything about a process when it's not running. Linux calls it `task_struct`.

Interview takeaway: Context switch overhead is why you don't want too many threads/processes — cache thrashing and TLB flushes hurt performance. This is why async I/O and event loops (Node.js, nginx) can outperform thread-per-request models at high concurrency.

4.6 Scheduling Algorithms

The scheduler decides which ready process runs next. Different goals: maximize CPU utilization, minimize wait time, maximize throughput, minimize response time, ensure fairness.

Key metrics:

- **Turnaround time** = completion time − arrival time
- **Waiting time** = turnaround − burst time
- **Response time** = time from arrival to first scheduled

- **Throughput** = processes completed per unit time

FCFS --- First Come First Served

Run processes in arrival order. Simple queue.

```
Processes: P1(burst=10), P2(burst=5), P3(burst=2)
Arrival:   P1 at 0,      P2 at 0,     P3 at 0

Gantt:
|——————————P1——————————|———P2———|—P3—|
0                          10         15   17

Waiting: P1=0, P2=10, P3=15 → Average = 8.33
```

Problem: Convoy Effect — one long job blocks all short jobs. Like a slow truck at a one-lane bridge.

Non-preemptive. No starvation (everyone eventually runs). Poor for interactive systems.

SJF --- Shortest Job First

Always run the job with the smallest CPU burst next. Optimal for minimizing average waiting time.

```
Same processes, SJF order: P3(2), P2(5), P1(10)

Gantt:
|—P3—|———P2———|——————————P1——————————|
0    2          7                          17

Waiting: P3=0, P2=2, P1=7 → Average = 3.0
```

Problem: Need to know burst time in advance (impossible in practice). Solved by **prediction** (exponential averaging of past bursts).

SRTF (Shortest Remaining Time First) = preemptive SJF. If a new job arrives with shorter remaining time, preempt current job.

Starvation risk: Long jobs may never run if short jobs keep arriving.

Round Robin (RR)

Each process gets a fixed time quantum (e.g., 10ms), then preempted and moved to back of ready queue. Fair, good for interactive systems.

```
Processes: P1(burst=10), P2(burst=6), P3(burst=3)  quantum=4

Gantt:
|——P1——|——P2——|——P3——|——P1——|——P2——|
0       4       8       11      15      17

P1 runs 0-4, P2 runs 4-8, P3 runs 8-11 (done), P1 runs 11-15 (done), P2 runs 15-17 (done)
```

Quantum size matters:

- Too small → too many context switches, overhead dominates
- Too large → degenerates to FCFS
- Typical: 10-100ms

No starvation. Good response time for interactive processes.

Priority Scheduling

Each process has a priority number. CPU goes to highest priority process. Can be preemptive or non-preemptive.

Starvation problem: Low-priority processes may never run. **Solution: Aging** — gradually increase priority of waiting processes (Linux: `nice` value adjustments).

MLFQ --- Multi-Level Feedback Queue

The most important real-world scheduling insight. Solves the "we don't know burst time" problem by learning from behavior.

```python
Queue 0 (highest priority, quantum=8ms):    [process enters here]
Queue 1 (medium priority, quantum=16ms):    [if uses full quantum, demotes here]
Queue 2 (lowest priority, FCFS):           [CPU-bound jobs end up here]
```

Rules:

1. New job enters highest priority queue
2. If job uses full quantum → demote to lower queue (it's CPU-bound)
3. If job gives up CPU before quantum expires → stay at same level (it's I/O-bound, interactive)
4. After some time period, boost all jobs back to top queue (prevents starvation)

Why this is genius: I/O-bound processes (interactive) naturally float to top. CPU-bound background jobs sink to bottom. System *learns* without being told burst times.

Interview takeaway: MLFQ is what real OSes approximate. It distinguishes interactive (short bursts, I/O-bound) from batch (long bursts, CPU-bound) jobs dynamically.

CFS --- Completely Fair Scheduler (Linux)

Linux's actual scheduler since kernel 2.6.23. Instead of fixed time quanta, it tracks **virtual runtime (vruntime)** per task and always runs the task with the smallest vruntime.

Key ideas:

- Every task gets an equal share of CPU time (weighted by priority/niceness)
- Uses a **red-black tree** (sorted by vruntime) to find next task in O(log n)
- Target latency: try to give every task CPU within some period (e.g., 20ms)
- Minimum granularity: won't preempt if task has run less than 0.75ms
- Nice values adjust weight: `nice -20` gets ~10x more CPU than `nice +19`

```
red-black tree (sorted by vruntime):

        [P3: vrt=100]
       /             \
[P1: vrt=50]    [P5: vrt=200]
      \
   [P2: vrt=80]

Scheduler picks leftmost node (P1, smallest vruntime) → runs it →
updates P1's vruntime → reinsert → tree rebalances
```

vruntime adjusts for priority: a high-priority task's vruntime increases slower, so it gets scheduled more often.

CFS groups (cgroups integration): You can assign CPU shares to groups of processes — this is how containers (Docker) limit CPU usage.

Scheduling Algorithm Comparison Table

Algorithm	Preemptive	Starvation	Convoy	Best For	Drawback
FCFS	No	No	Yes	Batch jobs	Convoy effect

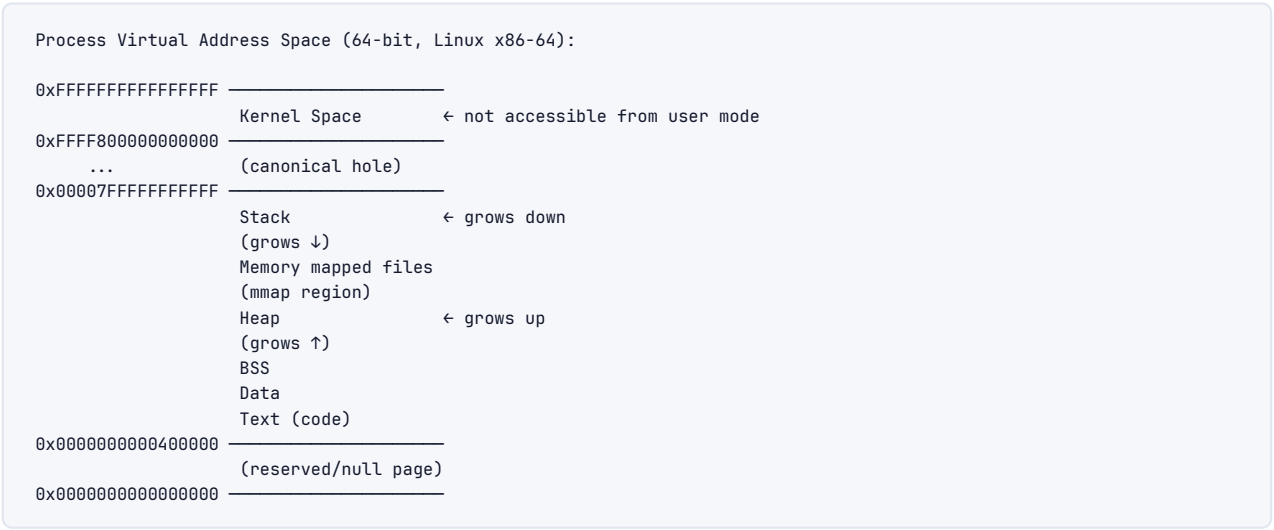
Algorithm	Preemptive	Starvation	Convoy	Best For	Drawback
SJF (non-preempt)	No	Yes	No	Min avg wait	Needs burst time
SRTF	Yes	Yes	No	Min avg wait	Needs burst time
Round Robin	Yes	No	No	Interactive	Context switch overhead
Priority	Both	Yes	No	Varied needs	Starvation
MLFQ	Yes	No (aging)	No	General purpose	Complex tuning
CFS	Yes	No	No	Linux general	Complex impl

4.7 Virtual Memory

The key insight of virtual memory: **every process believes it has the entire address space to itself**. The OS + hardware hardware translates virtual addresses to physical addresses.

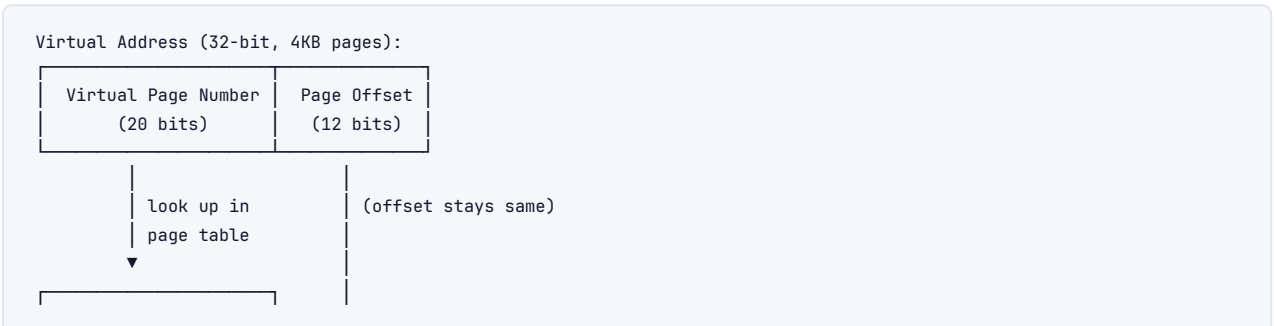
Benefits:

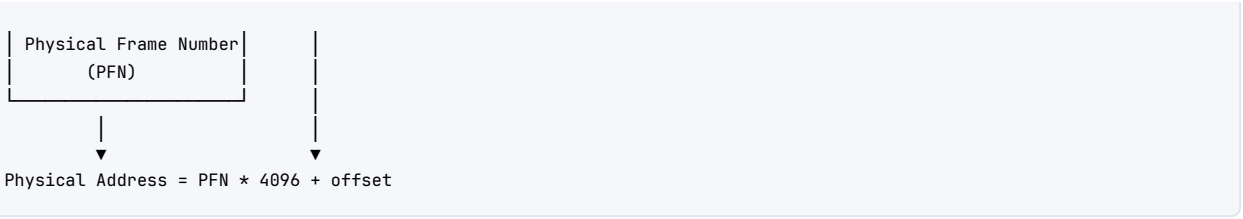
1. **Isolation** — process A can't read process B's memory (different page tables)
2. **More memory than RAM** — use disk as extension (swap)
3. **Simplified loading** — programs can be compiled to fixed virtual addresses
4. **Shared memory** — two processes can map the same physical page (e.g., shared libraries)
5. **Demand paging** — only load pages actually accessed



4.8 Paging

Paging divides virtual and physical memory into fixed-size **pages** (typically 4KB). The OS keeps a **page table** that maps virtual page numbers (VPN) to physical frame numbers (PFN).





Page Table Entry (PTE) contains:

- Physical Frame Number
- Present bit (is page in RAM?)
- Dirty bit (has page been written?)
- Accessed bit (has page been read recently?)
- Read/Write/Execute permissions
- User/Supervisor bit

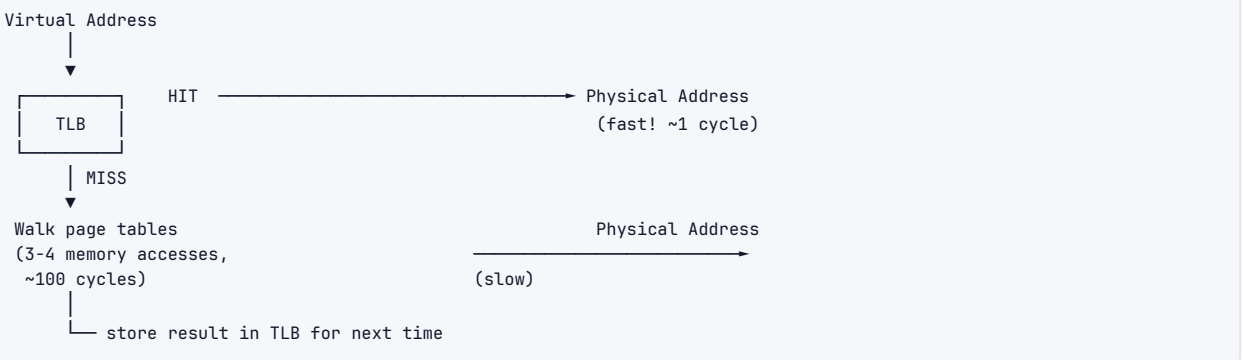
Problem with single-level page table: For 32-bit address space with 4KB pages: 2^20 = 1M entries, each 4 bytes = 4MB **per process**. With 1000 processes = 4GB just for page tables. Too much.

Solution: Multi-level page tables

x86-64 uses 4-level page tables: PML4 → PDP → PD → PT → Physical (each 9 bits = 512 entries, 1 bit unused, 12-bit offset)

4.9 TLB --- Translation Lookaside Buffer

Page table walks are expensive (3-4 memory accesses for multi-level tables). The **TLB** is a small, fast hardware cache of recent VPN → PFN translations.



TLB miss rates matter. If your program has good **spatial/temporal locality**, TLB hit rate is high (>99%). Programs with random access patterns across large memory regions thrash the TLB.

TLB flush: On context switch (new process = new page table = all TLB entries invalid), the OS must flush the TLB. This is expensive. Solutions:

- **ASID (Address Space ID):** Tag TLB entries with process ID, avoid full flush
- **Hardware TLB walkers:** Modern CPUs walk page tables in hardware (x86)

Huge pages (2MB or 1GB instead of 4KB): Fewer TLB entries needed for same memory coverage. Linux `hugepages`, `madvise(MADV_HUGEPAGE)`. Databricks and JVMs tune this.

Interview takeaway: TLB is why locality matters. Random memory access = TLB misses = slowness. Huge pages reduce TLB pressure for large working sets.

4.10 Page Faults

A **page fault** occurs when a process accesses a virtual address that doesn't have a valid PTE pointing to RAM.

Three types:

1. **Minor page fault** — page is in memory but PTE not set up (e.g., first access after mmap). No disk I/O. Fast.
2. **Major page fault** — page is on disk (swap or file). Must load from disk. Slow (~10ms).
3. **Invalid fault (segfault)** — address is not mapped at all, or permissions violated. Process gets SIGSEGV.

```
Process accesses virtual address 0x40001000
        |
        ▼
CPU checks TLB → MISS
        |
        ▼
CPU walks page table → Present bit = 0 (not in RAM)
        |
        ▼
Hardware raises Page Fault exception
        |
        ▼
OS page fault handler runs:
    Is address valid? → No → SIGSEGV → process dies
    Is address valid? → Yes:
        Minor fault? → set up PTE → return → retry instruction
        Major fault? → find page on disk → allocate physical frame →
                       read page from disk → set PTE → return → retry
        CoW fault? → copy page → update PTE → return → retry
```

Demand Paging: Pages are only loaded into RAM when accessed (on demand), not when process starts. Makes process startup fast. exec() just sets up page table entries pointing to the binary on disk; actual code pages load on first access.

4.11 Page Replacement Algorithms

When RAM is full and a new page must be loaded, the OS must **evict** an existing page. Which one?

FIFO --- First In First Out

Evict the oldest page in RAM.

```
Pages: 1 2 3 4 1 2 5 1 2 3 4 5  (reference string)
Frames: 3

FIFO:
1 → [1, -, -]   MISS
2 → [1, 2, -]   MISS
```

```
3 → [1, 2, 3]   MISS
4 → [4, 2, 3]   MISS (evict 1)
1 → [4, 1, 3]   MISS (evict 2)
2 → [4, 1, 2]   MISS (evict 3)
5 → [5, 1, 2]   MISS (evict 4)
...  9 page faults total
```

Belady's Anomaly: Adding more frames can sometimes cause MORE page faults with FIFO. Counterintuitive and bad.

Optimal (OPT)

Evict the page that will be used furthest in the future. Impossible to implement (requires future knowledge). Used as theoretical benchmark.

Achieves minimum possible page faults. Compare other algorithms against OPT to measure how close they are.

LRU --- Least Recently Used

Evict the page that hasn't been used for the longest time. Approximates OPT well in practice (temporal locality: recently used pages likely to be used again).

```
Same reference string, 3 frames, LRU:
1 → [1]        MISS
2 → [1,2]      MISS
3 → [1,2,3]    MISS
4 → [4,2,3]    MISS (1 LRU, evict 1)
1 → [4,1,3]    MISS (2 LRU, evict 2)
2 → [4,1,2]    MISS (3 LRU, evict 3)
5 → [5,1,2]    MISS (4 LRU, evict 4)
...  8 page faults (better than FIFO)
```

True LRU implementation: hardware timestamp per page, or doubly-linked list + hash map. Expensive.

Clock Algorithm (Second Chance)

Practical approximation of LRU. Each frame has a **reference bit** (set by hardware on access). A ”clock hand” sweeps through frames:

- If reference bit = 1: clear it (give second chance), advance hand
- If reference bit = 0: evict this page

```
Clock hand cycles through frames:
┌──────────────────────────────────┐
│  [P1,R=1] → [P2,R=0] → [P3,R=1]  │
│      ↑                           │
│   clock hand                     │
│                                  │
│  Visit P1: R=1, clear to 0, advance │
│  Visit P2: R=0, EVICT P2, done   │
└──────────────────────────────────┘
```

Linux uses a variant: pages have active/inactive lists, with the clock-like mechanism managing them.

Page Replacement Comparison Table

Algorithm	Fault Rate	Overhead	Belady's Anomaly	Notes
FIFO	High	Low	Yes	Simple, bad

Algorithm	Fault Rate	Overhead	Belady's Anomaly	Notes
Optimal	Lowest	Impossible	No	Theoretical benchmark
LRU	Low	High (true impl)	No	Best practical choice
Clock	Low-Medium	Low	No	Linux approximation
LFU	Medium	Medium	No	Frequency, not recency
Random	High	Negligible	No	Surprisingly decent

4.12 Thrashing

Thrashing occurs when a process (or system) spends more time paging than executing.

```
Scenario: 5 processes each need 4 frames. Only 10 frames total.

Each process gets 2 frames → working set doesn't fit in 2 frames
→ constant page faults → page fault handler runs → page fault handler
needs to evict pages that processes need → more page faults → ...

CPU utilization:
  100% ┤
       │        ╭─────╮
       │       /       \
   50% ┤      /         \
       │     /           \   ← THRASHING ZONE
    0% ┤────/             ──────────
       └────┴──────────────────────
            few       many
       (# of processes / degree of multiprogramming)
```

Working Set Model: Each process has a **working set** — the set of pages it actively uses in some time window. If the sum of all working sets > RAM, thrashing occurs.

Solutions:

- Reduce degree of multiprogramming (suspend some processes)
- Add more RAM
- Better page replacement (keep working sets in RAM)
- Local page replacement (each process has its own frame quota)

4.13 Segmentation

An alternative to paging: divide address space into variable-size **segments** (code, stack, heap, data). Each segment has a base address and limit.

```
Segment Table:
Segment | Base    | Limit  | Permissions
--------┼---------┼--------┼------------
Code    | 0x40000 | 0x2000 | R-X
Data    | 0x42000 | 0x1000 | RW-
Stack   | 0xFF000 | 0x3000 | RW-

Virtual address: (segment#, offset)
→ check offset < limit (else segfault)
→ physical = base + offset
```

Fragmentation problem: Variable-size segments cause external fragmentation — holes of unusable memory between segments.

Modern systems use **segmentation + paging** (x86-64 still technically has segments but with flat 0-base model, effectively disabled). Segments provide the logical view; paging handles the physical allocation.

4.14 Memory Management

Physical memory management: The kernel uses a **buddy system** allocator to manage physical frames. Free memory is maintained in lists of power-of-2-sized blocks. Allocation finds the smallest block that fits; deallocation merges adjacent buddies.

Virtual memory management (in process):

- **Kernel allocator:** `kmalloc()` for small kernel objects; `vmalloc()` for large non-contiguous virtual memory
- **User space:** `brk()`/`sbrk()` to extend heap; `mmap()` for arbitrary mappings
- **C library malloc:** sits above `brk()`/`mmap()`, maintains free lists, handles small allocations efficiently

Memory mapped files (mmap):

```
File on disk ──mapped to──► Region of virtual address space
                            Reads/writes go directly to page cache
                            Lazy loading (pages fault in on demand)
                            Shared: multiple processes map same physical pages
```

Used for: loading executables (ELF segments), shared libraries (every process maps libc), database buffer pools, Spark memory management.

4.15 IPC --- Inter-Process Communication

Processes are isolated. To communicate, they need OS-provided channels.

Pipes

Unidirectional byte stream. Parent → child (or chain of processes in shell).

```
ls | grep foo | wc -l

   ls        grep       wc
 stdout → | → stdin  stdout → | → stdin  stdout
          pipe1              pipe2
```

- **Anonymous pipes:** `pipe()` syscall. Only between related processes (parent-child).
- **Named pipes (FIFOs):** Exist as filesystem entries. Unrelated processes can use them.
- **Limitation:** Unidirectional, byte-stream only, local machine only.

Shared Memory

Fastest IPC. Two processes map the same physical pages into their address spaces.

```
Process A addr space:    Process B addr space:
0x1000 ──────────┐        0x2000 ──────────┐
                 └──► [Physical frames] ◄──┘
                        (shared pages)
```

- `shmget()` / `shmat()` (SysV) or `mmap()` with `MAP_SHARED`
- **No kernel involvement in data transfer** — just write to your virtual address, B sees it immediately
- **But:** Need synchronization (mutexes, semaphores) to avoid races
- Used by: databases (PostgreSQL shared buffer pool), media apps, high-frequency trading

Message Queues

Kernel-maintained queue of messages. Producers send messages; consumers receive them. Messages have types (for filtering).

- POSIX: `mq_open()`, `mq_send()`, `mq_receive()`
- Preserves message boundaries (unlike pipes which are byte streams)
- Can have multiple producers/consumers

Sockets

Most flexible IPC. Can be:

- **Unix domain sockets:** Local machine, file-system path, very fast
- **TCP/UDP sockets:** Network, across machines

Used for: nginx ↔ PHP-FPM, Docker daemon, database connections, literally all networked services.

Signals

Asynchronous notification to a process. `kill(pid, SIGTERM)`, `kill(pid, SIGKILL)`, etc. Limited: only carry the signal number, no data payload. Used for process control, not data transfer.

IPC Comparison Table

Method	Speed	Data Size	Synchronization	Scope	Use Case
Pipe	Fast	Stream	Blocking by default	Local (related)	Shell pipes, parent-child
Named Pipe	Fast	Stream	Blocking	Local	Unrelated local processes
Shared Memory	Fastest	Large	Manual (semaphores)	Local	High-perf data sharing
Message Queue	Medium	Messages	Built-in	Local	Producer-consumer patterns
Unix Socket	Fast	Stream/Dgram	N/A	Local	Service-to-service local
TCP Socket	Medium	Stream	N/A	Network	Distributed systems
Signal	Very fast	None (signal#)	N/A	Local	Process control

4.16 Deadlocks

A **deadlock** occurs when a set of processes are each waiting for resources held by another process in the set — a circular wait with no exit.

Coffman's 4 Necessary Conditions (all must hold for deadlock):

1. **Mutual Exclusion** — resource can only be held by one process at a time
2. **Hold and Wait** — process holds resources while waiting for more
3. **No Preemption** — resources can't be forcibly taken from a process
4. **Circular Wait** — P1 waits for P2 waits for P3 waits for P1

Mnemonic: **MHNC** — "My Habits Negatively Create" deadlocks
Deadlock Handling Strategies:

Strategy	Approach	Cost	Used When
Prevention	Eliminate one of Coffman's 4 conditions	May reduce concurrency	Design time
Avoidance	Banker's algorithm — only allocate if safe state	Overhead per allocation	Known max resources
Detection + Recovery	Let it happen, detect cycles, kill/rollback	Detection overhead	Databases (timeout + retry)
Ignore (Ostrich)	Pretend deadlocks won't happen	None	Linux/Windows (rare deadlocks → reboot)

Banker's Algorithm (Avoidance): Before granting a resource request, simulate the allocation. Check if a "safe sequence" still exists (an ordering where all processes can eventually complete). If yes → grant. If no → make process wait.

Deadlock Prevention techniques:

- Break Mutual Exclusion: use lock-free data structures
- Break Hold and Wait: acquire all locks at once (or release before requesting more)
- Break No Preemption: forcibly take resources back (works for preemptable resources like CPU)
- Break Circular Wait: impose a total ordering on lock acquisition (always acquire L1 before L2) — **this is the most practical technique**

Livelock: Processes actively respond to each other but make no progress. Like two people in a hallway both stepping aside in the same direction.

Starvation: A process never gets resources because others keep getting priority.

Interview takeaway: Prevent circular wait by enforcing lock ordering. In practice, databases detect deadlocks and roll back one transaction. OS mostly ignores deadlocks (reboot).

4.17 File Systems

A file system organizes data on persistent storage into a hierarchy of files and directories.

Key abstractions:

- **File:** Named sequence of bytes with metadata
- **Directory:** Special file that maps names to file metadata
- **inode:** Data structure storing file metadata

Inode

The **inode** (index node) is the core metadata structure for a file:

```
Inode #47:
┌──────────────────────────────┐
│ File type: regular file      │
│ Permissions: rw-r--r--       │
│ Owner: uid=1000, gid=1000    │
│ Size: 4096 bytes             │
│ Timestamps: atime/mtime/ctime│
│ Link count: 2                │
│ Direct blocks: [101, 102, 0] │ ← point to 4KB data blocks
│ Indirect block: 200          │ ← points to block of pointers
│ Double indirect: 300         │
│ Triple indirect: 0           │
└──────────────────────────────┘
```

Inode does NOT store the filename. The directory maps filename → inode number.

```
Directory "/home/user/":
┌──────────────────────┐
│ "." → inode 10       │
│ ".." → inode 5       │
│ "file.txt" → inode 47 │
│ "docs" → inode 52    │
└──────────────────────┘
```

Hard link: Two directory entries pointing to the same inode. Deleting one entry doesn't delete the file (link count decrements; file deleted when count reaches 0).

Soft link (symlink): A file that contains the path to another file. Independent inode. Becomes ”dangling” if target is deleted.

Block Addressing in ext2/ext3/ext4

```python
File data access via inode:

Direct pointers (12 blocks × 4KB = 48KB)
Single indirect (1 block of 1024 pointers × 4KB = 4MB)
Double indirect (1024 × 1024 × 4KB = 4GB)
Triple indirect (1024^3 × 4KB = 4TB)

For small files: only direct blocks needed (fast)
For large files: traverse indirect blocks (more disk accesses)
```

Modern ext4 uses **extents** instead: a range of contiguous blocks (base_block + length). Much more efficient for large files.

Journaling

Problem: Writing a file requires multiple disk writes (update inode, update data blocks, update directory). If system crashes mid-way, file system is inconsistent.

Solution: Journaling. Before making actual changes, write the intended changes to a **journal (log)**. After journal write completes (atomic), apply changes. If crash during apply, replay journal on recovery.

```
Write operation:
1. Write to journal (metadata or metadata+data)
   - journal_start
   - new inode
   - new data blocks
   - updated directory
   - journal_commit
2. Apply journal to actual disk locations
3. Mark journal entry as done (free space)

Crash recovery:
- If crash before journal_commit: abandon (no partial change made)
- If crash after journal_commit but before step 3: replay journal
```

Journal modes in ext4:

- **Writeback:** Only journal metadata. Fast but data might be stale after crash.
- **Ordered:** Journal metadata; data written to disk before journaling metadata. Safe, default.
- **Journal:** Journal both metadata and data. Slowest but safest.

Interview takeaway: Journaling prevents file system corruption on crash by ensuring operations are atomic. ext4 uses ordered mode by default.

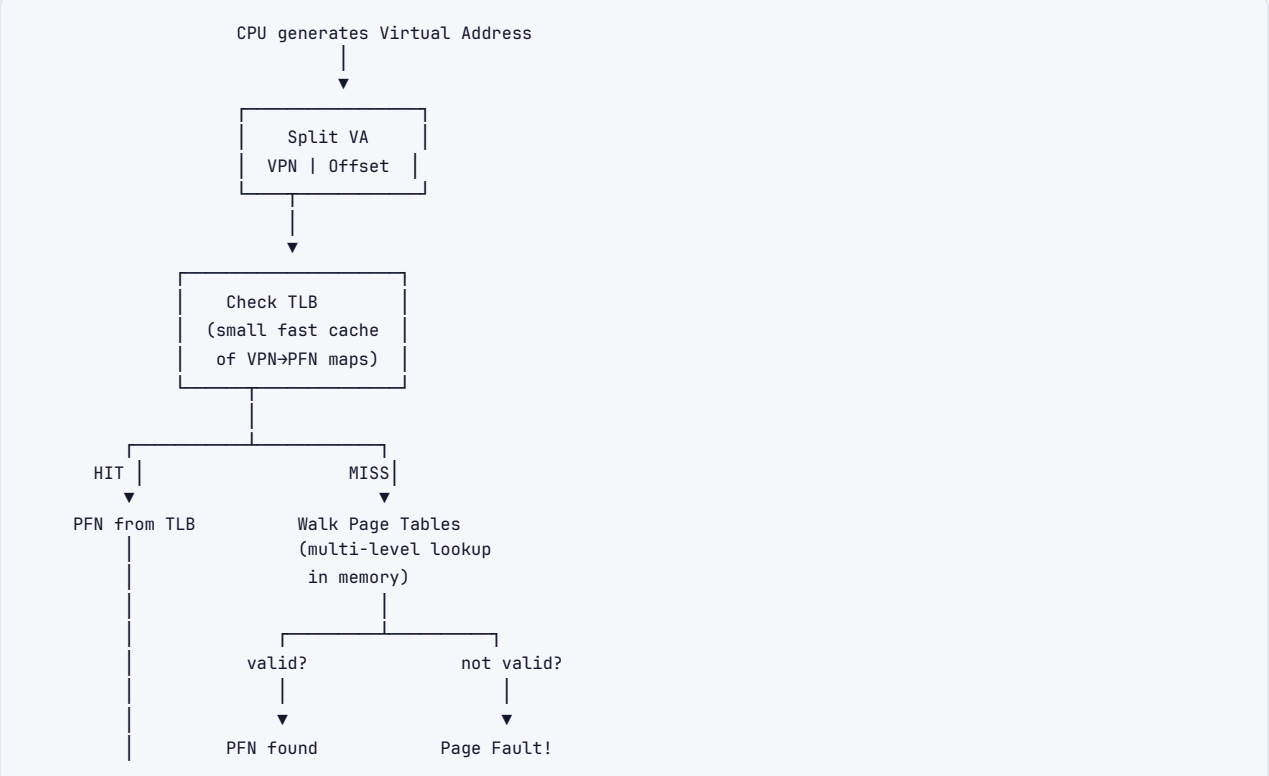
4.18 System Calls Deep Dive

Common system calls and what they do:

Category	Syscall	Description
Process	`fork()`	Clone current process
Process	`exec()`	Replace process image with new program
Process	`exit()`	Terminate process
Process	`wait()`	Wait for child to exit
Process	`getpid()`	Get process ID
Memory	`brk()`	Extend/shrink heap
Memory	`mmap()`	Map file/anonymous memory
Memory	`munmap()`	Unmap memory region
File	`open()`	Open file, return fd
File	`read()`	Read from fd
File	`write()`	Write to fd
File	`close()`	Close fd
File	`stat()`	Get file metadata
IPC	`pipe()`	Create pipe
IPC	`socket()`	Create socket
IPC	`send()/recv()`	Socket I/O
Sync	`futex()`	Fast userspace mutex (basis for pthreads)

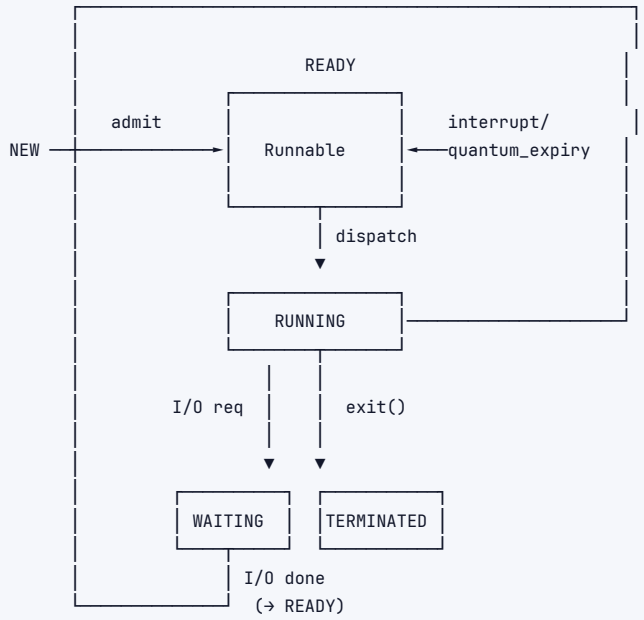
5 Architecture / Diagrams

5.1 Complete Virtual → Physical Address Translation



```
          │         update TLB          OS handles:
          │              │              - Load from disk
          │              │              - SIGSEGV
          │              │
          └──────────────┘
                 │
                 ▼
        Physical Address = PFN × pagesize + offset
                 │
                 ▼
            Access RAM
```

5.2 Process State Machine



5.3 Memory Layout: Kernel vs User Space

```
64-bit Linux Address Space:

0xFFFFFFFFFFFFFFFF ──────────────────────── (top)
                   [Kernel Space: 128TB]
                   - Kernel code & data
                   - Physical memory mappings
                   - Kernel stacks
0xFFFF800000000000 ──────────────────────
                   [Non-canonical hole]
                   (hardware enforced,
                    accessing = fault)
0x00007FFFFFFFFFFF ──────────────────────
                   [User Space: 128TB]
                   │
                   ├─ Stack (grows ↓)
                   │  [random stack base, ASLR]
                   │
                   ├─ mmap region (shared libs, mmap'd files)
                   │  libc.so, libpthread.so, etc.
                   │
                   ├─ Heap (grows ↑)
                   │  [malloc'd memory, managed by ptmalloc]
                   │
                   ├─ BSS (zero-initialized globals)
```

```
                  ├─ Data (initialized globals)
                  └─ Text (code, read-only)
                     [typically at 0x400000]
0x0000000000000000 ──────────────────────
                  [NULL page: unmapped, catch null deref]
```

5.4 Scheduling Gantt Charts Comparison

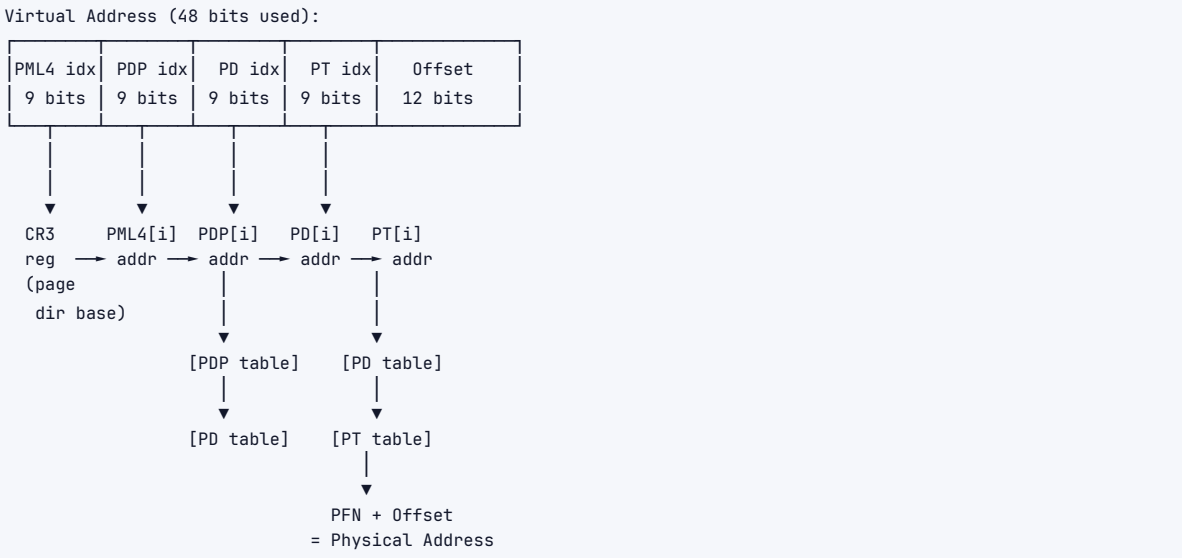
```
Processes: P1(0,10), P2(0,5), P3(0,3)  [arrival time, burst]

FCFS:
0    5   10   15   17
|──P1──────|──P2──|P3|
Avg wait: (0+10+15)/3 = 8.33

SJF (non-preemptive):
0  3  8          17
|P3|──P2──|──────P1──|
Avg wait: (0+3+8)/3 = 3.67

Round Robin (quantum=3):
0  3  6  9  12 14 17
|P1|P2|P3|P1|P2|P1|P2|
(P3 finishes at 9, P1 gets slice 9-12, P2 gets 12-14, P1 finishes 14-17, P2 finishes 17+)
```

5.5 Page Table Walk (4-level x86-64)



6 Real-World Examples

Chrome's Process-per-Tab Model: Google Chrome isolates each tab in its own process. If one tab crashes (or gets exploited), it can't affect other tabs or the browser. This is a deliberate security and reliability choice leveraging OS process isolation. Each tab has its own virtual address space, preventing cross-tab memory corruption.

Fork in Redis: Redis uses `fork()` to create background snapshots (RDB saves). The parent continues serving requests; the child writes memory to disk. Copy-on-write means the child sees a consistent snapshot without copying all data upfront. Writes from the parent after fork() cause CoW copies. Heavy write workloads during snapshots can double Redis's memory usage.

PostgreSQL Shared Memory: PostgreSQL uses shared memory (via `mmap()` or SysV shm)

for its buffer pool — pages from the database files are cached here, shared across all backend worker processes. Worker processes are separate processes (not threads), so they need IPC to share the buffer pool.

Linux Containers (Docker) = Namespaces + cgroups:

- **Namespaces:** Isolate what a process can see (PID namespace, network namespace, mount namespace). A container thinks it's the only process on the machine.
- **cgroups:** Limit how much resource a group of processes can use (CPU, memory, I/O). Fundamental OS mechanism.
- Virtual memory concepts directly apply: container memory limits translate to cgroup memory limits, causing page eviction or OOM kills.

JVM Garbage Collection interacts with OS VM: The JVM manages its own heap (Java objects), but that heap is RAM from the OS. GC pauses correlate with page faults if JVM heap pages were swapped out. This is why JVMs disable swap (`vm.swappiness=0`) on production servers. Major GC runs may trigger `madvise(MADV_DONTNEED)` to return pages to OS.

7 Real-Life Analogies

One building, one facilities team — every concept is a floor, a room, or a routine in the same office.

OS Concept	Real-Life Analogy
Process	A company renting a full floor — its own offices, filing cabinets, and staff, physically walled off from the companies on other floors; one company can't wander into another's space
Thread	An employee on that rented floor — shares the floor's filing cabinets and supplies with colleagues, but has their own desk, their own to-do list, and their own chair
Virtual Memory	Every employee gets a floor map labelling their rooms 1, 2, 3 … The building manager secretly maps those numbers to real rooms scattered across many floors — the employee sees a tidy sequence; the building is a patchwork
Page Table	The building manager's master ledger — one row per virtual room number, recording which actual room on which actual floor it corresponds to; without the ledger, no one can find anything
TLB	The receptionist's quick-reference card of the dozen rooms she looks up every hour — she checks the card first; only if the room isn't on it does she page through the full master ledger
Page Fault	You walk to a room listed on your map and the files aren't there — someone has boxed them and sent them to the basement archive (disk). A facilities runner fetches the box (major fault); if the map just forgot to update after a recent delivery, the runner corrects the entry on the spot (minor fault)
Context Switch	The building has one big glass-walled meeting room. When Team A's booking ends, they pack every whiteboard note and laptop into labelled boxes; facilities wheels in Team B's boxes and pins their diagrams back up before the next session starts
Deadlock	Team A is holding the sole projector and waiting for the laser pointer that Team B is holding — and Team B is waiting for the projector. Both teams sit in their rooms, arms folded, waiting forever
Scheduling (RR)	The facilities team rotates access to the one shared high-speed printer: each department gets a 10-minute slot, then the next department's key card is activated, cycling through the list fairly
MLFQ	New print jobs enter the express tray (highest priority, short slot). If a job hogs the printer through its full slot, it gets moved to the standard tray, then to the overnight bulk tray — quick one-page memos naturally stay at the top; the thousand-page report drifts to the bottom

OS Concept	Real-Life Analogy
Fork + CoW	A company on floor 3 wants to spin up a branch office (child process). Facilities hands the branch a copy of the floor map pointing to the same filing cabinets. The moment the branch edits a document, facilities quietly makes a private copy of that cabinet drawer — until then, nothing is duplicated
Pipe	The inter-floor mail chute: Team A drops documents in at the top, Team B collects them at the bottom; flow is strictly one-way and the chute can only hold so many envelopes before it backs up
Shared Memory	A glass-fronted document wall in the lobby that both the 3rd-floor and 5th-floor companies can read and update directly — the fastest way to exchange data, but teams must agree on who writes when or pages get overwritten
File System Inode	The building's asset tag record for a cabinet drawer: it doesn't hold the documents, but it records owner, creation date, number of folders inside, and exactly which shelf bays to find them in — the drawer's name is only in the floor directory, not on the record itself
Journaling	Before facilities moves a cabinet from one room to another, the facilities manager writes the move order in the shift log. If the power cuts out mid-move, the morning crew reads the log and either completes or reverses the move — nothing is left half-done
Thrashing	The building has 10 rooms but 15 teams each need 4 rooms simultaneously. Facilities constantly evicts one team's boxes to make space for another — everyone spends the day waiting for their boxes to arrive from the basement rather than doing any actual work
Zombie Process	A company vacates its floor and returns the key, but facilities hasn't yet struck it from the tenant register. The floor is empty, yet the register still lists the company as an active tenant until the landlord (parent) signs off
Semaphor	The parking-garage gate counter — a sign shows spaces remaining. Each entering car decrements the count; the gate refuses entry at zero. Exiting cars increment it, and the next waiting car is waved in

8 Memory Tricks / Mnemonics

Coffman's Deadlock Conditions: "MHNC"

> **M**utual exclusion, **H**old and wait, **N**o preemption, **C**ircular wait "My Habits Negatively Create deadlocks"

Process States: "NR RWT"

> **N**ew → **R**eady → **R**unning → **W**aiting → **T**erminated "New Runners Really Want Trophies"

Scheduling Algorithm Priority (for interactive systems):

> "MLFQ > RR > Priority > SJF > FCFS" Best to worst for interactive responsiveness

Page Fault Types: "MMS"

> **M**inor (no disk), **M**ajor (disk access), **S**egfault (invalid) "Minor = Minor inconvenience, Major = Major disk trip, Segfault = process death"

Page Replacement — what to evict:

> **FIFO**: First In, First Out (oldest) **LRU**: Least Recently Used (longest unused) **OPT**: Optimal (furthest future use) **Clock**: LRU approximation (second chance) Memory trick: "FLOC" — First, Least, Optimal, Clock

IPC Methods by Speed:

> Shared Memory > Pipe/Socket > Message Queue > Signal (for data capacity) "Sharing Privately Moves Slowly" (Shared > Pipe > Message > Signal)

TLB Miss Cost:

> Hit: ~1 cycle. Miss: ~100 cycles. Page fault: ~10ms (10 million cycles) "1, 100, 10M" — each step is 100x worse

Fork vs Exec:

> fork = **F**ission (split into two) exec = **Ex**change (swap out current program)

Kernel vs User Mode:

> K**ernel**: **K**ings can do everything User: **U**ntrusted, **U**nrestricted access denied

Buddy Allocator:

> "Buddies pair up" — adjacent same-size free blocks merge into bigger block

9 Common Interview Questions

9.1 Q1: What is the difference between a process and a thread?

Model Answer: A process is an isolated instance of a running program with its own virtual address space, file descriptors, and OS resources. A thread is a unit of execution within a process — multiple threads share the same address space, heap, and file descriptors but each has its own stack and registers.

Key tradeoffs: processes provide strong isolation (crash in one doesn't affect others) but are heavier to create and communicate between. Threads are lightweight and share memory naturally, but a crash or corruption in one thread kills the whole process, and shared memory requires careful synchronization.

Follow-ups:

- When would you use processes vs threads? (Processes: fault isolation like Chrome tabs, different security contexts; Threads: shared data, parallel computation within a task)
- How does fork() work? What is copied? (Virtual address space via CoW, FDs, signal handlers; not memory locks or thread-specific state)
- What's a zombie process? (Child that exited but parent hasn't called wait())

9.2 Q2: Explain virtual memory and why it's useful.

Model Answer: Virtual memory gives each process the illusion of having the entire address space to itself. The OS + hardware maintains page tables that map virtual page numbers to physical frame numbers. When a process accesses a virtual address, hardware translates it to

a physical address (via TLB → page table).

Benefits: isolation (each process has its own mapping so can't read others' memory), ability to use more memory than physical RAM (swap), sharing (two processes can map the same physical pages), and demand paging (only load pages that are actually accessed).

Follow-ups:

- What happens on a page fault? (Check if valid → load from disk or allocate → update PTE → retry instruction)
- What is the TLB? Why does it matter? (Cache of recent VPN→PFN; without it, every memory access needs 3-4 memory lookups for page table walk)
- What is thrashing? (Working set exceeds RAM → constant paging → CPU spends more time on page I/O than actual work)

9.3 Q3: How does the OS choose which process to run next?

Model Answer: The scheduler picks from the ready queue based on policy. Common algorithms:

- FCFS: simple FIFO, suffers convoy effect
- Round Robin: time slices, fair for interactive
- SJF: optimal average wait but needs burst time knowledge
- MLFQ: practical real-world — runs I/O-bound (interactive) jobs at higher priority; CPU-bound jobs sink to lower queues
- CFS (Linux): tracks virtual runtime per task, always runs task with smallest vruntime, uses a red-black tree

Follow-ups:

- What's the convoy effect in FCFS? (One long job blocks all short jobs behind it)
- How does Linux CFS work? (vruntime, red-black tree, weighted by nice value)
- What's the difference between preemptive and non-preemptive scheduling? (Preemptive: OS can interrupt running process; non-preemptive: process runs until it yields or blocks)

9.4 Q4: Explain deadlocks --- how do they occur and how do you prevent them?

Model Answer: Deadlock requires all four Coffman conditions: mutual exclusion, hold-and-wait, no preemption, and circular wait. The most practical prevention technique is **lock ordering** — always acquire locks in a globally consistent order to eliminate circular wait. Databases typically use detection + timeout + rollback. OS largely uses the ostrich algorithm (ignore it; restart if needed).

Follow-ups:

- What's livelock? (Processes actively respond to each other but make no progress — like two people in a hallway always stepping in the same direction)
- What's the Banker's algorithm? (Safe-state analysis before granting resources; check if there exists a safe sequence; theoretical, rarely used in practice)
- How does a database detect deadlock? (Build a wait-for graph; detect cycles; roll back the youngest/cheapest transaction in the cycle)

9.5 Q5: What is a context switch and what makes it expensive?

Model Answer: A context switch saves the current process's CPU state (registers, PC, SP) to its PCB, runs the scheduler to pick the next process, loads that process's CPU state, and if it's a different process, switches the page table (CR3 register on x86) and flushes the TLB.

The expense comes from: (1) direct cost of saving/restoring registers (~microsecond), (2) TLB flush — new process has cold TLB so first memory accesses need full page table walks, (3)

cache pollution — new process's data evicts old process's hot data from CPU caches. This is why event-driven servers (nginx, Node.js) can outperform thread-per-request at high concurrency.

Follow-ups:

- Why are thread switches cheaper than process switches? (Same address space → no page table switch → no TLB flush → caches stay warm)
- What is an ASID (Address Space ID)? (Tag TLB entries with process identifier to avoid full TLB flush on context switch)

9.6 Q6: Explain how fork() and copy-on-write work.

Model Answer: fork() creates a child process that is an exact copy of the parent. Instead of physically copying all memory pages (which could be gigabytes), the OS uses copy-on-write: both parent and child initially share the same physical pages, marked read-only. When either process writes to a page, a page fault occurs, the kernel copies that specific page, and the writing process gets its own private copy. Only written pages are duplicated.

This makes fork() fast when followed immediately by exec() (child replaces its address space with a new program without ever writing to the CoW pages) and efficient in general since most data never gets written.

Follow-ups:

- Why is fork() + exec() two steps? (Unix philosophy: separation of process creation and program loading. Allows manipulating the process environment between fork and exec — e.g., redirect file descriptors.)
- What's vfork()? (Like fork but child borrows parent's address space entirely; parent is suspended; useful only if exec() is immediately called)
- How does Redis use fork() for RDB snapshots? (Fork creates child; child writes memory to disk while parent continues serving; CoW means parent's writes create new page copies, child sees original snapshot)

10 Senior-Level Discussion Points

1. Scheduler Tuning in Production

Linux CFS can be tuned via `/proc/sys/kernel/sched_*`. For latency-sensitive workloads (trading, real-time audio), you might use `SCHED_FIFO` or `SCHED_RR` policies with `RT_PRIORITY`. CPU pinning (`taskset`, `sched_setaffinity`) prevents context switches by keeping a process on one CPU — important for avoiding NUMA cross-socket memory access.

2. NUMA (Non-Uniform Memory Access)

In multi-socket servers, each CPU has local RAM (fast, ~50ns) and remote RAM on other sockets (slow, ~100ns). OS-level NUMA awareness means scheduling threads near their data. `numactl`, `mbind()`, `set_mempolicy()`. Databricks/Spark performance tuning involves NUMA-aware memory allocation.

3. Memory Overcommit

Linux by default overcommits memory — `malloc()` always ”succeeds” even if insufficient RAM. Pages are allocated lazily on write. When RAM is truly exhausted, the OOM (Out of Memory) Killer selects and kills a process. This is why a system can appear to have plenty of memory but processes still get killed. Controlled by `/proc/sys/vm/overcommit_memory`.

4. Huge Pages and TLB

For workloads with multi-GB working sets (databases, JVMs, key-value stores), 4KB pages require enormous TLB entries. 2MB huge pages reduce TLB entries by 512x. Linux Transparent Huge Pages (THP) automates this but can cause latency spikes (background defragmentation). Production databases often disable THP and manage huge pages manually.

5. io_uring (Modern Linux I/O)

Traditional I/O: `read()`/`write()` = system calls = context switches. `io_uring` (Linux 5.1+) allows batching many I/O operations in a ring buffer shared between kernel and userspace, reducing syscall overhead dramatically. Databases and web servers (nginx, Cloudflare) are adopting this.

6. Container and Kubernetes Scheduling

Kubernetes scheduler is an application-level scheduler on top of OS schedulers. A Pod is scheduled to a Node; the OS on that node schedules the container's processes. cgroups v2 provides hierarchical resource accounting. Understanding OS scheduling helps debug container performance issues (CPU throttling, memory limits causing OOM).

7. Spectre and Meltdown (Hardware + OS intersection)

These exploits used speculative execution to read across process boundaries, violating OS isolation guarantees. Mitigations (KPTI — Kernel Page Table Isolation) added overhead to system calls by unmapping kernel pages from user space page tables. Understanding the OS/hardware boundary is critical for security-sensitive systems work.

11 Typical Mistakes Candidates Make

1. Confusing process and thread isolation Saying ”threads are isolated from each other” is wrong. Threads share address space — one thread can corrupt another's stack or heap. Only processes are strongly isolated.

2. Thinking fork() physically copies memory It uses copy-on-write. Pages are shared until written. Don't say ”fork copies all memory.”

3. Confusing virtual and physical addresses When asked ”where is this variable in memory?”, the variable has a virtual address. The physical address depends on page table mapping and whether the page is even in RAM right now.

4. Ignoring TLB in context switch cost Candidates say ”context switch just saves registers.” The real cost is TLB flushing and cache misses for the new process's data. This is the actual performance bottleneck.

5. Saying deadlock happens with just circular wait Deadlock requires ALL FOUR Coffman conditions. Circular wait alone isn't sufficient.

6. Confusing scheduling algorithms' use cases Saying ”SJF is the best scheduler” — it's optimal for average wait time but impractical (requires knowing burst times) and causes starvation. Real systems use MLFQ/CFS.

7. Thinking page faults are always bad Minor page faults are normal and fast. Demand paging intentionally causes page faults on first access. Major page faults are the expensive ones (disk I/O). Segfaults kill the process.

8. Forgetting that zombie processes consume resources The PCB (task_struct) persists until the parent calls wait(). Many zombie processes can exhaust PID space.

9. Confusing pipes and sockets Pipes are unidirectional, local, byte streams. Sockets are bidirectional, can be networked, and have both stream (TCP) and datagram (UDP) modes.

10. Not knowing what system calls are expensive `getpid()` → cheap (cached). `open()` → moderate. `read()`/`write()` (especially small I/O) → expensive per call. Candidates should know to batch I/O, use buffering, and minimize syscall frequency.

12 How This Connects To Other Topics

12.1 Concurrency and Synchronization

OS provides the primitives (mutexes, semaphores, condition variables via `futex()`), but the synchronization problems (race conditions, deadlocks) are concurrent programming problems. OS scheduling makes races possible by preempting threads at any instruction. Understanding OS scheduling helps understand why race conditions are hard to reproduce (timing-dependent).

12.2 Performance Engineering

- CPU scheduling → latency and throughput of services
- Virtual memory/TLB → why memory access patterns matter (cache locality)
- Page faults → why initialization cost of large allocations is deferred
- Context switches → why event loops can beat threads at high concurrency
- Page cache → why disk I/O is faster than expected (kernel caches file blocks in RAM)

12.3 Databases

- File systems + journaling → database WAL (Write-Ahead Log) is the same concept at the application layer
- OS page cache → databases fight with OS cache (use `O_DIRECT` for their own buffer management)
- `mmap()` → some databases (SQLite, LMDB) use memory-mapped I/O
- Process model → PostgreSQL (process-per-connection) vs MySQL (thread-per-connection)
- Lock management → database locks are application-level, but OS provides the primitives

12.4 Cloud Computing

- Virtualization (EC2) → hypervisor virtualizes CPU + memory, same concepts as OS but one level up
- Containers → namespaces + cgroups, direct OS features
- Serverless (Lambda) → tiny VMs (Firecracker), OS startup cost matters
- Distributed systems → sockets are OS IPC extended over network; distributed consensus has analogies to OS concurrency control

12.5 Networking

- Sockets are OS abstractions over network hardware
- TCP buffers are kernel memory (sk_buff)
- `epoll` / `kqueue` → OS-level event notification for I/O-bound servers
- Socket options (`SO_REUSEPORT`, `SO_KEEPALIVE`, `TCP_NODELAY`) are kernel tuning knobs

13 FAANG Interview Tips

1. Lead with the ”why” before the ”what” Don't just define virtual memory — explain *why* it exists (isolation, overcommit, abstraction) before explaining *how* it works (page tables, TLB). Interviewers want to see systems thinking, not memorization.

2. Draw diagrams proactively For virtual address translation, process state machines, and scheduling Gantt charts — start drawing immediately. It shows structured thinking and makes complex topics instantly clearer.

3. Connect OS to real systems you've used ”Redis uses fork() for snapshots and CoW means...” shows you understand OS concepts in real context. FAANG interviewers value this highly.

4. Know the tradeoffs, not just the facts "Thread vs process" — you must know when to choose each, not just how they differ.

5. Be ready to go deep on one topic Often the interviewer will probe one area deeply. If they ask about virtual memory, be ready to go all the way down to TLB shootdowns or Spectre mitigations if asked.

6. Quantify costs

- System call: ~100ns
- Context switch: ~1-10μs (with cache effects)
- TLB miss: ~100ns
- Major page fault: ~10ms
- L1 cache hit: ~1ns, L2: ~4ns, L3: ~40ns, DRAM: ~100ns, SSD: ~100μs, HDD: ~10ms

7. Know the Linux specifics FAANG uses Linux everywhere. Know `task_struct`, CFS, cgroups, namespaces, procfs (`/proc/meminfo`, `/proc/[pid]/maps`). Mention Linux-specific terms when relevant.

8. Handling system design questions with OS flavor When designing a key-value store: mention page cache, mmap options, WAL for durability (= journaling). When designing a web server: mention epoll, process vs thread model, socket buffers. OS knowledge elevates system design answers.

9. Common Google-specific angles Google cares about: scalability, latency tails, resource efficiency. OS topics come up as: "how would you minimize context switches in a high-RPC system?", "explain how Borg schedules containers" (kernel scheduling at cluster scale).

10. Practice explaining to a rubber duck Pick a topic (say, "page table walk") and explain it out loud from scratch. If you can't, you don't know it well enough. FAANG interviews are verbal — clarity of explanation matters.

14 Revision Cheat Sheet

14.1 10-Minute Summary

The OS is a virtualization and protection layer. It multiplexes the CPU (scheduling), virtualizes RAM (virtual memory + paging), and isolates processes (separate address spaces, privilege levels).

Processes vs Threads: Process = isolated execution environment. Thread = lightweight execution unit within process. Choose process for isolation, thread for efficiency.

Scheduling: FCFS (simple, convoy problem) → RR (fair, quantum-based) → SJF (optimal wait, needs burst time) → MLFQ (practical, learns I/O vs CPU behavior) → CFS (Linux, vruntime + red-black tree).

Virtual Memory: Every process has a virtual address space. Page tables map virtual → physical. TLB caches recent translations. Page faults load missing pages from disk (major) or fix up mappings (minor).

Deadlocks: Need all 4 Coffman conditions. Prevent by enforcing lock ordering. Databases detect + rollback. OS ignores (ostrich).

IPC: Shared memory (fastest), pipes (simple), sockets (flexible, networked), message queues (bounded).

File Systems: Inode = metadata (not filename). Directories map names to inodes. Journaling = write intentions before acting, enables crash recovery.

14.2 Key Numbers

Metric	Value
Page size	4KB (typical)
TLB hit	~1 cycle
TLB miss (page walk)	~100 cycles
Context switch overhead	~1-10 μs
System call cost	~100-1000 ns
Major page fault (SSD)	~100 μs
Major page fault (HDD)	~10 ms
L1 cache	~1 ns
L3 cache	~40 ns
DRAM access	~100 ns

14.3 Compact Cheat-Sheet Table

Topic	Key Point	Interview Gotcha
Process vs Thread	Different address space vs shared	Threads not isolated from each other
fork()	CoW, not a full copy	Heavy writes after fork() = doubled memory
exec()	Replaces address space, same PID	Must follow fork() in typical usage
Zombie	Child done, parent hasn't wait()'d	Consumes PID and PCB
Virtual Memory	Illusion of full address space	Virtual $\neq$ physical
Page Table	Maps VPN $\rightarrow$ PFN	Multi-level to save space
TLB	Cache of VPN$\rightarrow$PFN	Flush on context switch (process)
Page Fault	Minor (fast), Major (disk), Segfault (die)	Minor faults are normal and expected
FCFS	FIFO, simple	Convoy effect
RR	Fair, quantum-based	Quantum size matters
SJF	Optimal wait time	Needs burst time, starvation
MLFQ	Learns I/O vs CPU dynamically	No starvation (aging)
CFS	vruntime + red-black tree	Linux actual scheduler
Deadlock	Need all 4 Coffman conditions	Circular wait alone $\neq$ deadlock
Lock ordering	Prevents circular wait	Most practical deadlock prevention
Shared memory	Fastest IPC	Needs explicit synchronization
Pipes	Unidirectional byte stream	Only between related processes
Inode	Metadata, not filename	Filename stored in directory entry
Journaling	Write intent before acting	ext4 ordered mode = default
Thrashing	Sum of working sets > RAM	Reduce multiprogramming degree
CoW	Shared pages until write	Redis fork() + heavy writes = 2$\times$ memory
Huge Pages	Reduce TLB pressure	THP can cause latency spikes
cgroups	Limit CPU/memory per process group	Foundation of containers

14.4 Most Important Concepts (Ranked by Interview Frequency)

1. **Process vs Thread** — asked in nearly every systems interview
2. **Virtual Memory + Paging + TLB** — foundational for everything
3. **Context Switch cost** — critical for performance discussions

4. **fork() + CoW** — specific, frequently tested
5. **Deadlock + prevention** — classic OS question
6. **Scheduling algorithms** — MLFQ and CFS are most important
7. **Page fault handling** — demand paging, CoW faults
8. **IPC mechanisms** — compared by speed and use case
9. **File system / inode** — storage system design questions
10. **System calls + kernel/user mode** — architecture foundation

15 Visual Reference

A set of figures distilling the key quantitative intuitions of this topic.

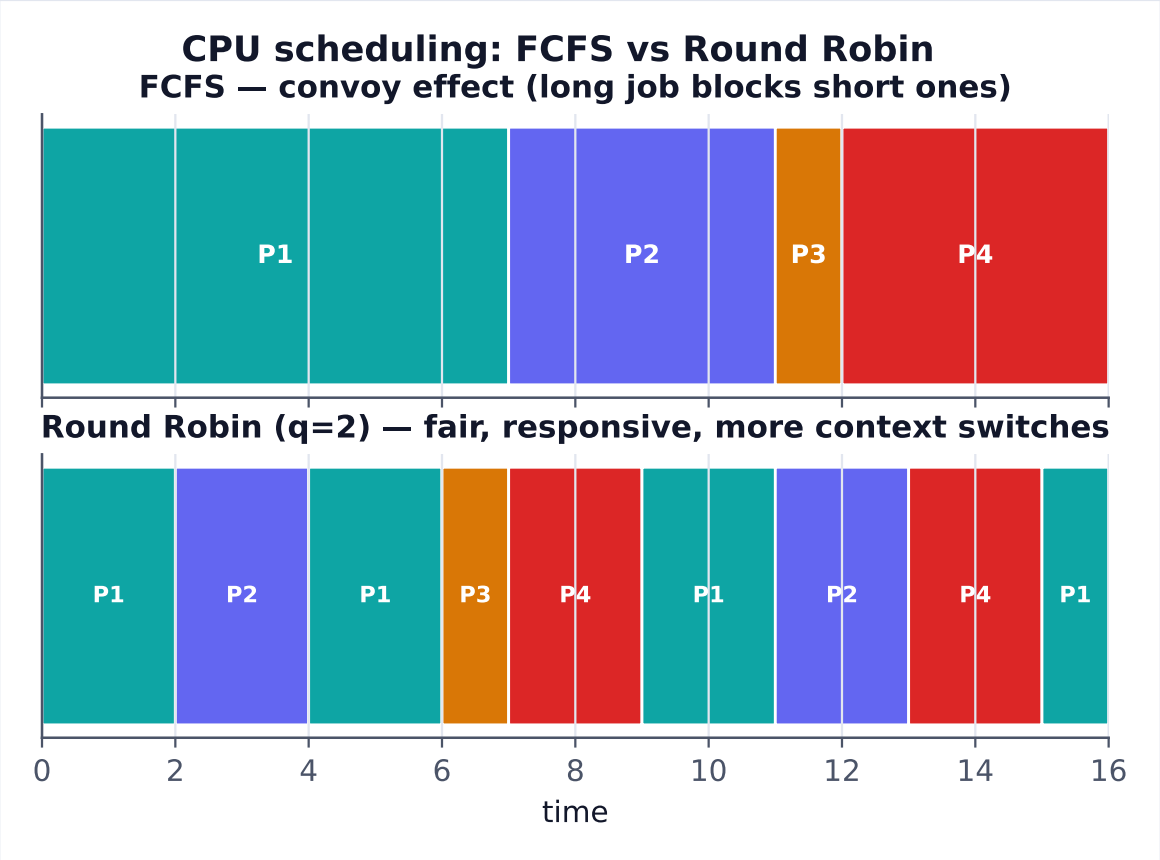


Figure 1. FCFS is simple but suffers the convoy effect; Round Robin time-slices for fairness and responsiveness at the cost of extra context switches.

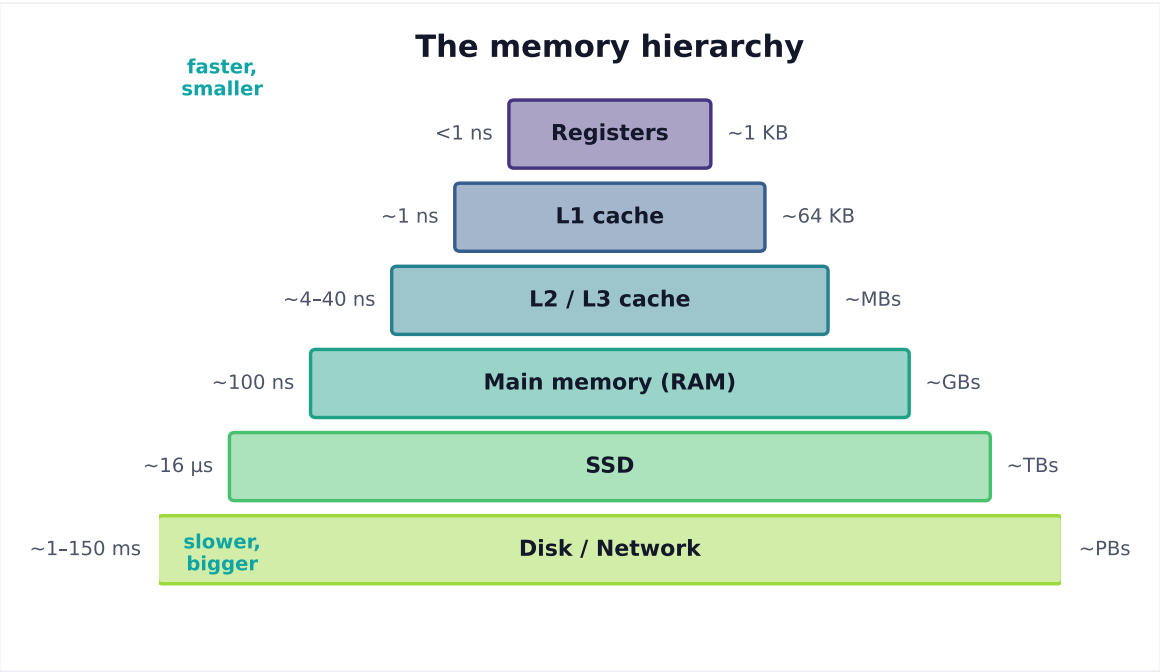


Figure 2. Each level trades capacity for speed by orders of magnitude. Performance comes from keeping the working set in the fastest tier that fits — the basis of caching.